

What is Re-render: সহজভাবে বললে **re-render** মানে **component**-এর **render logic** আবার **run** করা। কিন্তু "পুরো application আবার শুরু থেকে run করা" না।

```
import { use, useState } from "react"
import type { CountryType } from "../..type";
import { CountryCard } from "../CountryCard/CountryCard";
import "../countries.css"

interface CountriesProps {
  countriesPromise: Promise<CountryType[]>;
}

export default function Countries({ countriesPromise }: CountriesProps)
{
  const countries = use(countriesPromise);
  const [visitedFlag, setVisitedFlag] = useState<string[]>([]);

  const handleFlag = (flag: string): void => {
    if (visitedFlag.includes(flag)) {
      const reamaingFlags = visitedFlag.filter(elem => elem !==
flag); // React সাধারণত reference comparison ব্যবহার করে বুঝতে পারে state
change হয়েছে কিনা। তাই push করলে হবে না।
      setVisitedFlag(reamaingFlags); // setVisitedFlag শুধু
visitedFlag এর value পরিবর্তন করে না, React-কে re-render করার জন্যও জানায়।
    }
    else {
      const newVisitedFlag = [...visitedFlag, flag];
      setVisitedFlag(newVisitedFlag); // setVisitedFlag শুধু
visitedFlag এর value পরিবর্তন করে না, React-কে re-render করার জন্যও জানায়।
    }
  }

  return (
    <div>
      <p>Countries: {countries.length}</p>
      <h3>Total Flag(s) added: {visitedFlag.length}</h3>
      <div className="visitedFlags">
        {
          visitedFlag.map((elem, index) => <img key={index}
src={elem} alt="Visited Flag" />)
        }
      </div>
    </div>
  )
}
```

```

        <div className="countries">
          {
            countries.map(country => <CountryCard
              key={country.ccn3.ccn3}
              country={country}
              handleFlag={handleFlag}
              visitedFlag={visitedFlag}></CountryCard>)
          }
        </div>
      </div>
    )
  }
}

```

```

import { useState } from "react";
import type { CountryType } from "../type";
import "../countrycard.css"

interface CountryCardProps {
  country: CountryType,
  handleFlag: (flag: string) => void,
  visitedFlag: string[]
}

export function CountryCard({country, handleFlag, visitedFlag}:
CountryCardProps) {

  const [visited, SetVisited] = useState(false);
  const handleVisited = ()=>{
    SetVisited(!visited);
  }
  return (
    <div className={`countryCard ${visited ? 'country-visited' :
    ''}`}>
      <h3>{country.name.common}</h3>
      <img src={country.flags.flags.png}
alt="country.flags.flags.alt" />
      <p>Population: {country.population.population}</p>
      <p>Capital: {country.capital.capital}</p>
    </div>
  )
}

```

```

        <button onClick={handleVisited}>{visited ? "Visited" :
"Mark as Visited"}</button>
        <br />
        <br />
        { /* <button onClick={ () =>
handleFlag(country.flags.flags.png)}>Add Flag</button> */ }
        <button onClick={ () =>
handleFlag(country.flags.flags.png)}>{visitedFlag.includes(country.flag
s.flags.png) ? "Flag Added" : "Add Flag"}</button>
      </div>
    )
  }
}

```

এই example এ:

প্রথমবার:

visitedFlag = []

তখন **Countries()** run করে → **CountryCard** render হয়।

User click করলো:

setVisitedFlag(["bd.png"])

তারপর React বলবে:

"State change হয়েছে, এই component-এর UI আবার calculate করতে হবে।"

পুরো **Countries** component function-টাই আবার execute হবে।

তখন কী কী আবার **execute** হবে?

সহজভাবে:

Countries()

↓

const countries = use(...)

↓

const [visitedFlag, setVisitedFlag] = useState(...)

↓

handleFlag function তৈরি

↓

return-এর JSX calculate

↓

visitedFlag.length

↓

visitedFlag.map(...)

↓

countries.map(...)

↓

প্রতিটি CountryCard-এর নতুন props তৈরি

অর্থাৎ **Countries** component-এর **function body** আবার **execute** হয়।

যেহেতু **Countries** আবার এইটা তৈরি করছে:

```
<CountryCard
```

```
...
```

```
  visitedFlag={visitedFlag}
```

```
/>
```

তাই **CountryCard**-এর render-ও update হবে।

কিন্তু একটা গুরুত্বপূর্ণ **distinction**

Function পুরোটা **execute** হওয়া $\neq$ পুরো **UI/DOM/HTML** নতুন করে তৈরি হওয়া।

মানে:

```
function Countries() {  
  // এই component function-এর code আবার execute হয়  
  ...  
}
```

React প্রথমে নতুন UI representation **calculate/compare** করে, তারপর DOM-এ যেখানে দরকার শুধু সেখানে **update** করে। React শেষ পর্যন্ত DOM-এর শুধু প্রয়োজনীয় অংশ **update** করে। তাই React পুরো **DOM** মুছে আবার বানাবে না। শুধু প্রয়োজনীয় DOM changes করবে। Full DOM newly create হয় না।

তাই তোমার প্রশ্নের সরাসরি উত্তর:

Countries() component-এর **function body**-টাই আবার **execute** হয়। তারপর React নতুন result আগের result-এর সাথে মিলিয়ে প্রয়োজনীয় UI update করে।

mental model:

State change

↓

Countries() পুরো function আবার execute

↓

নতুন UI কী হওয়া উচিত React সেটা calculate করে

↓

আগের UI-এর সাথে compare করে

↓

যেখানে change হয়েছে শুধু সেটুকু DOM update করে